

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Computer Vision with OpenCV COURSE CODE: DSA02

COURSE OUTCOME COVERED

CO5: Apply computer vision techniques to real-world applications such as face recognition, tracking, medical imaging, and autonomous systems. (BL3)

Assessment Tool Weightage: 30%

QUESTIONS: 2

Total Marks:30

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I B.Deepthi-192424269 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B.Deepthi

Question 1: Face Detection and Recognition Using Eigenfaces

1. Problem Understanding

The aim is to develop an algorithm that can look at a grayscale image, find all the faces present in it, and identify the people by comparing their facial features with a database of known people.

Input

- A grayscale image of size $M \times N$.
- A database containing previously stored face images and their names.

Output

- Bounding box coordinates around every detected face.
- The recognized name corresponding to each detected face.

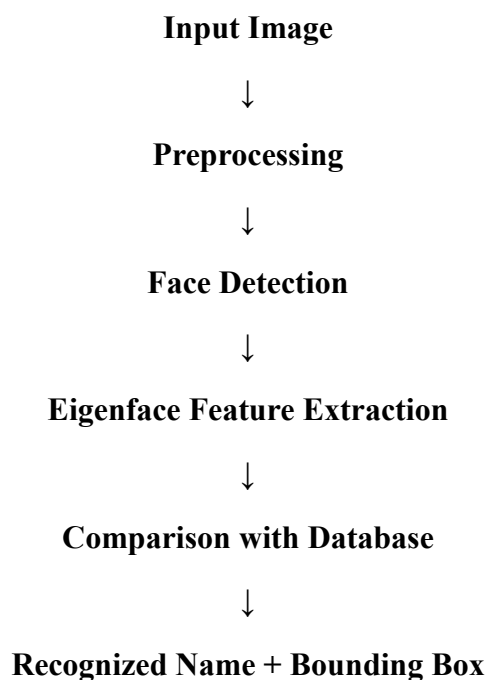
Main Constraints

- More than one person may appear in the same image.
- Lighting may be different from the images stored in the database.
- The algorithm should normalize the image before recognition.

The assessment specifically requires preprocessing, face detection, Eigenface feature extraction, Euclidean-distance comparison, and output of bounding boxes with labels.

2. Algorithm Design

The algorithm can be divided into five simple stages:



Step 1: Preprocessing

First, normalize the grayscale image. Histogram equalization is then applied to reduce the effect of uneven lighting.

Step 2: Face Detection

Use a **Viola-Jones detector** or sliding-window method to locate faces.

Step 3: Eigenface Feature Extraction

Each detected face is converted into an Eigenface feature representation.

Step 4: Recognition

The extracted feature vector is compared with the stored feature vectors of known people using **Euclidean distance**.

Step 5: Final Decision

The database entry having the smallest acceptable distance is selected as the recognized person.

If the distance is too large, the face should be labelled **Unknown** instead of forcing an incorrect identity.

3. Pseudocode

ALGORITHM Eigenface_Face_Recognition

INPUT:

Grayscale image I of size $M \times N$

Known face database D

OUTPUT:

Bounding boxes and names of detected faces

BEGIN

1. Read the grayscale image I
2. Normalize the intensity values of I
3. Apply histogram equalization to I
4. Detect all faces in I using Viola-Jones detector
5. IF no faces are detected THEN
 Display "No face detected"
 STOP
END IF

```

6. FOR each detected face F DO
    a. Crop the face region using its bounding box
    b. Resize F to the standard face size
    c. Convert F into Eigenface feature vector E
    d. Set minimum_distance = INFINITY
    e. Set recognized_name = "Unknown"
    f. FOR each known person K in database D DO
        i. Calculate Euclidean distance
           between E and K's Eigenface vector
        ii. IF distance < minimum_distance THEN
            minimum_distance = distance
            recognized_name = K's name
        END IF
    END FOR
    g. IF minimum_distance is greater than
       recognition_threshold THEN
        recognized_name = "Unknown"
    END IF
    h. Draw the bounding box around F
    i. Display recognized_name above the bounding box
END FOR

7. Display the final image
END

```

4. Control Structures Used

The algorithm uses control structures appropriately:

FOR Loop

A FOR loop is used to process every detected face.

FOR each detected face

This is important because the image may contain multiple people.

Nested FOR Loop

A second loop compares each detected face with every known person in the database.

FOR each known person

IF-ELSE Condition

Conditions are used to:

- Check whether a face was detected.
- Find the minimum Euclidean distance.
- Decide whether the person is known or unknown.

This makes the algorithm capable of handling different situations instead of assuming that every face will be recognized.

5. Completeness and Optimization

To make the algorithm more practical:

- Histogram equalization handles varying illumination.
- Multiple detected faces are processed individually.
- The nearest Eigenface match is selected.
- A recognition threshold prevents incorrect forced matches.
- Face images are resized before feature extraction to reduce computation.
- The database feature vectors can be calculated once and reused.

Example

Suppose an image contains **three people**:

Face 1 → Ravi

Face 2 → Unknown

Face 3 → Priya

The final output will contain three bounding boxes, with the corresponding names displayed above them.

Why This Algorithm Is Suitable

I would use this approach because Eigenfaces converts a large amount of facial pixel information into a smaller feature representation. This makes comparison simpler and faster. Histogram equalization also makes the system less sensitive to changes in lighting.

Question 2: Real-Time Road Lane Detection

1. Problem Understanding

The objective is to detect road lane markings continuously from a video captured by a front-facing camera and highlight those lanes on every frame.

Input

- Video frames captured from a front-facing camera.

Output

- The detected lane lines highlighted on each video frame.

Main Constraints

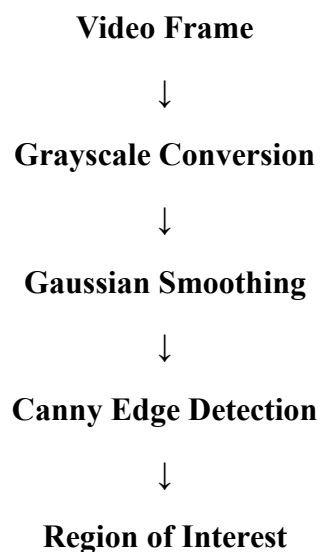
The system should continue working when:

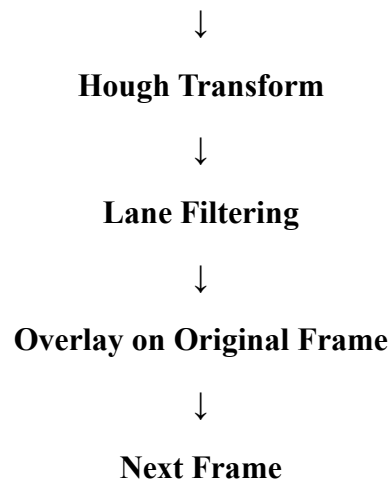
- Shadows are present.
- The road contains curves.
- Part of a lane is temporarily hidden or occluded.
- The video contains many consecutive frames.

The assessment specifies grayscale conversion, Gaussian smoothing, Canny edge detection, Hough Transform, and overlaying the detected lanes on the original frame.

2. Algorithm Design

The lane detection process can be represented as:





Step 1: Capture Frame

Read one frame from the front-facing camera.

Step 2: Convert to Grayscale

Convert the frame into grayscale because lane detection mainly requires intensity and edge information.

Step 3: Gaussian Smoothing

Apply Gaussian blur to reduce small noise and unnecessary edges.

Step 4: Canny Edge Detection

Detect strong edges that may correspond to lane markings.

Step 5: Region of Interest

Focus mainly on the road area instead of processing the sky, buildings, and other irrelevant regions.

Step 6: Hough Transform

Use the Hough Transform to identify lines from the detected edges.

Step 7: Lane Selection

Filter detected lines according to their position and orientation so that irrelevant lines are ignored.

Step 8: Overlay

Draw the selected lane lines on the original frame.

Step 9: Repeat

Continue this process until the video ends.

3. Pseudocode

ALGORITHM Real_Time_Lane_Detection

INPUT:

Video stream V from a front-facing camera

OUTPUT:

Video frames with detected lane lines highlighted

BEGIN

1. Open the video stream V
2. WHILE video stream is available DO
 - a. Capture the current frame F
 - b. Convert F into grayscale image G
 - c. Apply Gaussian smoothing to G
to obtain smooth image S
 - d. Apply Canny edge detection to S
to obtain edge image E
 - e. Select the road region of interest
from E
 - f. Apply Hough Transform to detect
possible lane line
 - g. FOR each detected line L DO
 - IF L satisfies lane position
and orientation conditions THEN
 - Keep L
 - ELSE
 - Ignore L
 - END IF
 - END FOR
 - h. IF valid lane lines are found THEN
 - Draw the lane lines on F

ELSE

Display the original frame

END IF

i. Display the processed frame

j. Move to the next video frame

END WHILE

3. Release the video stream

4. Close all display windows

END

4. Handling Shadows, Curves and Partial Occlusion

Shadows

Shadows can create unwanted edges. To reduce this problem, the algorithm should focus on the expected road region and use appropriate edge thresholds.

Curved Roads

A basic Hough Transform works best for straight lines. For curved roads, the detected lane points can be fitted using a polynomial curve.

For example:

$$x = ay^2 + by + c$$

This allows the detected lane to follow a curved road instead of assuming that every lane is perfectly straight.

Partial Occlusion

If a vehicle temporarily covers part of a lane, the visible portion of the lane can still be detected. Previous-frame lane information can also be used to estimate the missing portion.

This makes the algorithm more stable than treating every frame independently.

5. Control Structures Used

WHILE Loop

The WHILE loop continuously processes frames while the video is available.

WHILE video stream is available

This is essential for real-time video processing.

FOR Loop

A FOR loop checks each detected line.

FOR each detected line

IF-ELSE

Conditions are used to:

- Check whether a line is a possible lane.
- Decide whether valid lanes were detected.
- Handle frames where lane detection fails.

Therefore, the algorithm can respond to different road situations rather than producing an output blindly.

6. Completeness and Optimization

For a more reliable real-time system:

1. Use a **Region of Interest** to reduce unnecessary processing.
2. Apply Gaussian smoothing before Canny detection.
3. Tune Canny thresholds according to road conditions.
4. Filter Hough lines based on their slope and location.
5. Store lane information from previous frames.
6. Use polynomial fitting for curved roads.
7. Avoid processing irrelevant portions of the image.
8. Display the original frame if lane detection temporarily fails.

Example

If the camera captures a road where the left and right lanes are visible:

Camera View

$$\begin{array}{c} \backslash \quad / \\ \backslash \quad / \\ \backslash \quad / \\ \backslash \quad / \\ \backslash \quad / \\ \vee \end{array}$$

Detected Road Lanes

The algorithm identifies the lane edges and draws them over the original video frame.

Final Comparison of the Two Algorithms

Feature	Eigenface Face Recognition	Lane Detection
Input	Grayscale image	Video frames
Main task	Detect and recognize faces	Detect road lanes
Preprocessing	Normalization + histogram equalization	Grayscale + Gaussian blur
Main technique	Eigenfaces	Canny + Hough Transform
Comparison/Detection	Euclidean distance	Line/curve filtering
Control structure	Nested loops + conditions	Video loop + line filtering
Special cases	Multiple faces, unknown person, lighting	Shadows, curves, occlusion
Output	Face box + person's name	Highlighted lane lines

Conclusion

Both algorithms follow the same basic computer-vision idea: **prepare the input, extract useful visual information, make a decision, and produce a meaningful output.**

For face recognition, the important challenge is making facial features reliable despite lighting and multiple people. For lane detection, the challenge is continuously finding meaningful road boundaries despite noise, shadows, curves, and temporary obstruction.

The pseudocode therefore does more than list OpenCV operations. It includes **conditions, loops, exception handling, and practical decision-making**, which makes the algorithms suitable for real-world implementation and satisfies the assessment requirements for algorithm design and completeness.